

LaserTank

Technical Documentation

*A puzzle game in C++ (SFML 2.6) with an online score board
Overview of the source code in src/ and include/*

Document version: 1.1

Date: July 2026

Contents

1. Introduction
2. How the game starts
3. Project architecture
4. Application lifecycle
5. The game loop (GameLoop)
6. The map and the tile system
7. The signal system
8. The player (tank) and the bullet
9. Enemy tanks
10. The undo mechanism
11. Game state: PlayerState and MapState
12. Level loading and networking
13. Scores and the server
14. File overview
15. Notes and possible improvements

1. Introduction

LaserTank is a puzzle game played on a 16 by 16 grid. The player drives a tank that moves across the tiles, fires bullets, and tries to reach the flag (the goal) on each level. Along the way there are walls that can be destroyed, mirrors that deflect the bullet, movable blocks, water, transport tracks, and enemy tanks that fire at the player whenever the player lines up with them.

The project is written in C++ with the SFML 2.6 library for graphics, sound, and the window. Levels are downloaded from a server in the TMX format (Tiled maps), and scores (the move count) are sent to and read from the same server through libcurl.

This document describes the source code in the `src/` and `include/` directories. The goal is to explain how the code is organized, which classes exist, and how they cooperate during a single game.

Core features

- A 16 by 16 grid, where a single tile is 32 pixels.
- Several tile types: walkable ground, destructible wall, indestructible wall, mirrors, water, movable block, flag, transport tracks, and enemy tanks.
- The player moves with the arrow keys, fires with Space, and undoes a move with U.
- An undo system that remembers both the player moves and the state of the whole map after firing.
- Downloading levels from a remote server and sending scores (the move count) to a high score table.

2. How the game starts

The entry point of the program is the main function in `src/main.cpp`. It is deliberately short: it creates an Application object, calls its run method, and catches any exception.

```
int main() {
    try {
        Application app;
        return app.run();
    } catch (const std::exception& e) {
        std::cerr << "[FATAL] " << e.what() << '\n';
        return -1;
    }
}
```

All of the high level logic lives in the Application class. This keeps main simple, and any hard failure (an exception while loading a texture, a map, and so on) is reported to the user instead of the program crashing without a message.

3. Project architecture

The code is split into clearly separated parts. Instead of one large class that does everything, the work is broken into smaller managers, each with a single responsibility. MainGame ties them all together and hands them references to the shared objects (the player, the map, the window).

Part	Files	Role
Application	application/	Main program flow, menu, moving between levels.
Game	game/MainGame	Assembles all subsystems for a single level.
Loop	game/GameLoop	Event handling, updates, and rendering per frame.
Input	game/inputHandler	Translates keys into moves and firing.
Bullets	game/bulletManager	Tracks active bullets and detects hits on the player.
Enemies	game/enemyManager	Decides when an enemy tank fires.
Undo	game/undoManager	Undoing moves and map states.
State	game/gameStateChecker	Checks whether the level is won or lost.
Rendering	game/renderer	Draws the map, the player, the bullets, and the HUD.
Map	map/	The tile grid: movement, mutation, animation.
Player	player/	The tank, firing, pushing blocks.
Tiles	include/*Tile.h	The behaviour of each kind of tile through signals.
Network	network/, sendToServer, getScores	Level download and score handling.

A key idea in the design is that MainGame owns all of the main objects as its own fields, and the managers receive references to them through the constructor. That way every part works on the same data, with no copying and no global variables for the game state.

4. Application lifecycle

The constructor of the Application class (application/application.cpp) prepares everything that is needed before the game begins:

- Initializes libcurl (curl_global_init).
- Downloads all available levels from the server (downloadAllLevels).
- Loads the global font and all textures (loadGlobalFont, loadTextures).
- Counts how many map files exist and prepares the per level score table.

The run method drives the main flow. First the main menu is shown. If the user closed the window or chose nothing, the program exits cleanly. Otherwise it enters a loop that, for the chosen level, creates a MainGame object and runs it.

After each game Application reads the result:

- If Back was clicked, the menu opens again.
- If the window was closed, the application exits.
- If the level was won, handleLevelWon runs (it records the score, sends it to the server, shows the table) and the game moves on to the next level.

Advancing to the next level uses the remainder of the division $(\text{chosenLevel} + 1) \% \text{levelCount}$, so after the last level the sequence wraps back to the start.

5. The game loop (GameLoop)

GameLoop (game/GameLoop.cpp) is the heart of a single game. As long as the window is open, on every pass it checks whether the level is finished. If it is not, a normal frame is processed; if it is, the loop switches to the end screen (win or loss).

A normal frame

The processNormalFrame method does the following steps:

- After a set amount of time it enables key repeat, so holding an arrow key moves the tank several times.
- Updates the water and enemy tank animations (updateAnimations).
- Handles window events: closing, key presses, and mouse clicks on the Undo and Back buttons (handleWindowEvents).
- Checks the tile the player stands on through PlayerInteraction (for example a transport track moves the player).
- If the player has a bullet, it calls BulletInteraction to handle the bullet colliding with tiles.
- If the conditions allow it, it asks EnemyManager whether an enemy tank should fire.
- Updates all bullets (bulletManager.update) and draws the scene (renderer.render).

Animations

updateAnimations uses two clocks (sf::Clock). When the given time passes, the water switches frame (the ripple), and the enemy tanks cycle through three tread frames. This gives the impression of a living world without a separate thread for animation.

The end screen

processEndGameFrame is called when the game is over. On a win, a sound plays and a window opens for the player to enter their initials. On a loss, the Game Over sound plays and a choice is offered: undo the move (Undo) and continue, or restart. When undoing, the bullet state is cleared and the previous position is restored, so the game continues as if the last fatal move never happened.

6. The map and the tile system

The Map class (include/map.h) represents the 16 by 16 grid. It holds two parallel views of the same map:

- tileMap: a matrix of integers, where each number is the code of a tile type (see the code table).

- tiles: a matrix of smart pointers (unique_ptr) to Tile objects, which carry the graphics and behaviour.

The integer view is easy to check quickly and to store as state (undo), and the object view carries the behaviour of each tile. When a tile changes, both views are updated.

So that the map file does not grow huge, the Map class is split by responsibility across several source files:

File	Contents
map/map.cpp	Constructor, buildMap (building tiles from numbers), drawing.
map/mapQuery.cpp	Queries: whether a field is walkable, a tile position, a state snapshot (getMapState).
map/mapMovement.cpp	Moving tiles (moveTile) and updating transport tracks.
map/mapMutation.cpp	Changing the map: destroying a wall, destroying a tank, undoing the whole map.
map/mapAnimation.cpp	Switching the water and enemy tank frames.

Tile codes

The codes are defined in include/tileCode.h as the TileCode enum. The same numbers are used both in the TMX level files and in the internal map matrix.

Code	Name	Meaning
0	PLAYER_START	The player start position (treated as walkable ground).
1	WALKABLE_GROUND	Walkable ground.
3	DESTRUCTIBLE_BLOCK	A wall that a bullet destroys.
4-7	MIRROR_1..4	Mirrors that deflect the bullet at an angle.
8	WATER	Water; the tank cannot enter it, a block sinks into it.
9	MOVABLE_BLOCK	A block that can be pushed and pushed into water.
10	FLAG	The flag, the goal of the level.
11	UNDESTRUCTIBLE_BLOCK	An indestructible wall.
12-15	ENEMY_TANK_*	An enemy tank (left/right/up/down).
16-19	DESTROYED_TANK_*	A destroyed tank (remains as an obstacle).
20-23	TRANSPORT_TRACK_*	A transport track that carries the player in a direction.
50	TILE_IN_WATER	A block sunk into water (now walkable across).

The same file also holds helper functions such as tileIsTransportTrack, tileIsEnemyTank, tileIsPlayerWalkable, and tileMapPositionInBounds. They describe the rules in one place (for example what the player can walk on), so the same check is not rewritten across the code.

TileFactory

The tile factory (src/tileFactory.cpp) takes a code and coordinates and returns a ready unique_ptr to the matching tile object with its texture already set. The whole link between a number and a tile class is in one place, which makes adding a new tile type easier.

7. The signal system

Instead of the main code checking the type of every tile and deciding what to do, each tile answers for itself what should happen. This is solved through two signal enums.

Bullet signals (TileSignal)

When a bullet enters a field, `tile->sendSignal(hit)` is called, where `hit` carries the direction and displacement of the bullet. The tile returns one of the signals, and `BulletInteraction` carries it out:

Signal	Effect
BLOCK_BULLET / DESTROY_BULLET	The bullet is removed (it hit an indestructible wall).
DESTROY_TILE	The tile is destroyed and becomes walkable ground.
MOVE_TILE_*	A block or tank is pushed in the given direction.
REFLECT_*	A mirror deflects the bullet into a new direction.
DESTROY_TANK_*	An enemy tank was hit on its weak side and destroyed.

Player signals (PlayerTileSignal)

When the player steps onto a field, `sendSignal()` is called with no arguments. The result tells `PlayerInteraction` what the field does to the player:

Signal	Effect
MOVE_PLAYER_*	A transport track carries the player in the given direction.
KILL_PLAYER	The field kills the player.
BLOCK_PLAYER	The field cannot be passed (a block, a mirror, and so on).
NONE	No effect (walkable ground).

The advantage of this approach is that the behaviour of a new tile is added by writing a new class that returns the right signal, without touching the bullet or player logic.

8. The player (tank) and the bullet

The `Tank` class (`include/player.h`, `src/player/`) represents the player. It stores the direction the tank faces, its grid position (`gridPos`), the sprite, and the sounds for moving, turning, and firing. The player also holds its bullet as a `unique_ptr`, which means the bullet is released automatically when it leaves the screen or hits an obstacle.

Moving and turning

The movement logic is in `PlayerInteraction` (`src/playerInteraction.cpp`). When an arrow key is pressed, it first checks whether the tank already faces that direction. If it does not, the tank first turns (one move), and only the next press moves it. Before moving, it checks whether the target field is inside the map and whether it is walkable.

Every move is recorded into the `playerStates` list (the direction and position before the move), which undo later uses to step the tank back.

Firing and pushing

When the player presses Space, `deleteOrMoveAdjBlockIfExists` is called first. If a block that can be pushed or destroyed sits in front of the tank, that is done right away and no bullet is fired. Otherwise a new bullet is created (`fireBullet`) that moves in the direction of the tank.

The bullet (`Bullet`, `src/bullet.cpp`) is a rectangle with a velocity vector in one of four directions. The player bullet moves a little slower than the enemy one (a factor of $1/1.5$ against 1), which can be seen in the update method.

9. Enemy tanks

An enemy tank is the `EnemyTank1` class (`include/tank1.h`). Each tank has a direction it faces and a weak side (`weakSide`) opposite to its barrel. A bullet destroys it only if it hits the weak side; from other directions the bullet merely pushes the tank.

When an enemy fires

`EnemyManager` (`game/enemyManager.cpp`) in the `shouldEnemyFireBullet` method checks all tanks in the same row and the same column as the player. For each tank `killPlayer` is called, which looks at whether there is a clear line between the tank and the player (only walkable fields or water). If there is, the tank fires a bullet toward the player.

When a tank fires, its bullet is added to the `BulletManager`, and the coordinates of the tank that fired are recorded (`coordXKillerTank`, `coordYKillerTank`). `GameLoop` uses those coordinates to decide when it is safe for the player to act again.

`BulletManager` (`game/bulletManager.cpp`) in the update method moves all bullets, checks whether a bullet has reached the player field (`killPlayer` is then set), and removes bullets that have left the map.

10. The undo mechanism

`UndoManager` (`game/undoManager.cpp`) makes it possible to undo a move. The key idea is that every move is marked with a kind of action, so that when undoing it is clear what needs to be restored.

- For every move a label is written into the `tankMovedOrBulletShot` list: "tank moved" or "bullet shot".
- Moving and turning the tank is restored from the `playerStates` list of the `Tank` class itself.
- Firing can change the map (a wall destroyed, a block pushed), so a full snapshot of the map is saved in `mapStates` and restored on undo.

When `undoMove` is called, it looks at the last action. If it was firing, the last saved map state is restored. If it was moving, the previous position and direction of the tank are restored. The move count is also decreased. This system is used both during the game (the U key or the Undo button) and on the loss screen, where the player can undo the fatal move and continue.

11. Game state: PlayerState and MapState

Deciding whether a level is won or lost is split into two small classes that each answer one question. Both live in the state namespace, in the files `src/game/PlayerState.h` and `src/game/MapState.h`. They are used by `GameStateChecker`, which now only combines their answers.

Note that the state namespace is used on purpose. The `include/player.h` header already holds a separate `PlayerState` struct that is only a per move snapshot (direction and position) used by `undo`. The namespace keeps these two names apart with no collision.

PlayerState

`PlayerState` answers whether the player is still alive. It holds references to the tank and the bullet manager, because the player can die in two ways: by stepping onto a deadly tile, or by being hit by a bullet.

Method	Meaning
<code>steppedOnDeadlyTile()</code>	True when the tile the player stands on marked it for death (<code>player.getShouldKillPlayer</code>).
<code>hitByBullet()</code>	True when a bullet has reached the player field (<code>bulletManager.isKillPlayer</code>).
<code>isDead()</code>	True when either of the two above is true.
<code>isAlive()</code>	The opposite of <code>isDead()</code> .

```
bool PlayerState::isDead() const {
    return steppedOnDeadlyTile() || hitByBullet();
}

bool PlayerState::isAlive() const {
    return !isDead();
}
```

MapState

`MapState` answers a question about the map under the player: which tile is beneath the tank and whether it is the goal. It holds references to the tank and the map, and reads the integer view of the map for the player position.

Method	Meaning
<code>tileUnderPlayer()</code>	Returns the tile code at the current player position.
<code>reachedGoal()</code>	True when the tile under the player is the flag (the goal).

```
int MapState::tileUnderPlayer() const {
    sf::Vector2i pos = player.getGridPosition();
    return tileMap.getTileMapInt()[pos.y][pos.x];
}

bool MapState::reachedGoal() const {
```

```
    return tileUnderPlayer() == GOAL_TILE;
}
```

How GameStateChecker uses them

GameStateChecker no longer holds the logic itself. It owns one PlayerState and one MapState and forwards the questions to them, which keeps each concern in its own place.

```
bool GameStateChecker::gameLost() const {
    return playerState.isDead();
}

bool GameStateChecker::gameWon() const {
    return mapState.reachedGoal();
}

bool GameStateChecker::gameEnd() const {
    return gameLost() || gameWon();
}
```

12. Level loading and networking

Levels are stored as TMX files (the Tiled format, XML) in the maps/ directory. Downloading from the server is done by levelDownloader (src/network/levelDownloader.cpp).

- downloadLevel(i): if the map already exists locally, it is skipped; otherwise it is fetched from the server through libcurl and saved as maps/mapN.tmx. If the download fails, the unfinished file is removed.
- downloadAllLevels(): downloads levels in order (1, 2, 3, ...) until one fails, which marks the end of the level list.

Parsing the map is done by readMapFromXML (src/extractMatrix.cpp) with the tinyxml2 library. It reads the layer with CSV encoding, splits the numbers by comma, and fills the 16 by 16 matrix in order. If the file does not exist or has no map element, an exception is thrown which main catches.

13. Scores and the server

When the player finishes a level, the move count is sent to the server and the high score table is shown.

- getScores.cpp: the getPlayerScores method downloads the JSON with the scores and extracts the name, score, and level for the requested level. If there is no internet connection, noInternetConnection is set.
- sendToServer.cpp: the sendMoveCount method sends the player name and move count as JSON to a PHP script through a POST request.

All network communication goes through the libcurl library, which is initialized at the start (curl_global_init) and cleaned up at the end (curl_global_cleanup) in the Application class.

14. File overview

The following table gives a short overview of the most important source files to make it easier to find your way around.

File	Role
src/main.cpp	Entry point, creates Application.
src/application/application.cpp	Main flow, menu, level progression, scores.
src/game/MainGame.cpp	Assembles the subsystems for a single level.
src/game/GameLoop.cpp	The game loop, events, frames, end screen.
src/game/inputHandler.cpp	Keys into moves and firing, link to undo.
src/game/bulletManager.cpp	Updating bullets and hits on the player.
src/game/enemyManager.cpp	The decision to fire an enemy.
src/game/undoManager.cpp	Undoing moves and map states.
src/game/gameStateChecker.cpp	Combines PlayerState and MapState for win/loss.
src/game/PlayerState.cpp	Checks whether the player is alive or dead.
src/game/MapState.cpp	Checks the tile under the player and the goal.
src/game/renderer.cpp	Drawing the scene and the HUD.
src/map/*.cpp	The tile grid: building, queries, movement, mutation, animation.
src/player/*.cpp	The tank, firing, pushing blocks.
src/playerInteraction.cpp	Player movement and reaction to a tile.
src/bulletInteraction.cpp	The bullet colliding with tiles.
src/tileFactory.cpp	Building a tile from a code.
src/bullet.cpp	The bullet (movement, speed, direction).
src/extractMatrix.cpp	Reading a TMX map into a matrix.
src/network/levelDownloader.cpp	Downloading levels from the server.
src/getScores.cpp / sendToServer.cpp	Reading and sending scores.
include/*Tile.h, tank1.h ...	Tile classes and their behaviour (signals).
include/tileCode.h	Tile codes and helper checks.

15. Notes and possible improvements

While reading the code, a few places stand out that work correctly but could be polished in future versions:

- Enemy tanks hold their bullets as raw pointers (Bullet*), unlike the player which uses a unique_ptr. Moving to smart pointers would make memory handling consistent.
- The bullet bounds check in places uses 16 as the upper limit of a field, while the grid is indexed from 0 to 15. It is worth checking that all bounds conditions are consistent.
- The updateWaterTiles method is empty; it is most likely a leftover from an earlier version and can be removed.
- The score parsing in getScores relies on a fixed line layout in the JSON. A proper JSON parser would be more resistant to format changes.

These are minor items and do not affect gameplay. They are listed for completeness and as pointers for further work on the project.

End of document